A 62.8 TOPS/W FP-INT Digital Computing-in Memory Processor with Bit-Reordered Adder Tree and Low Active Hierarchical Accumulator

Sunjoo Whang, Sangwoo Ha, Soyeon Um, Sangjin Kim and Hoi-Jun Yoo
KAIST, Daejeon, Republic of Korea
Email: sjwhang@kaist.ac.kr

2025 IEEE International Symposium on Circuits and Systems (ISCAS) | 979-8-3503-5683-0/25/$31.00 ©2025 IEEE | DOI: 10.1109/ISCAS56072.2025.11043177

***Abstract*—This paper introduces an energy-efficient DRAM-based FP-INT Digital Computing-in-Memory (CIM) processor for data-intensive AI workloads, addressing challenges in mixed precision, excess power consumption, and area overhead. The processor features three key innovations: 1) Bit-reordered adder tree (BRAT) that reduces adder bit width through exponent-aware reordering, achieving power and area reductions by 23% and 56%, respectively; 2) Low active hierarchical accumulator (LAHA) that conditionally updates the FP accumulator, minimizing accumulator activity by 57%; and 3) Speculative input block skipping (SIBS) to avoid unnecessary computations, reducing energy consumption by 15%. Implemented in 28nm CMOS technology, the processor achieves 62.8 TOPS/W in FP8 activation and INT4 weight precision for GPT2-Small inference. Our results show an overall energy savings of 31%, making this architecture highly efficient for modern large language models.**

***Index Terms*—DRAM, Computing-in-memory (CIM), FP-INT**

I. Introduction

Large language models (LLMs) like GPT and LLaMA have shown remarkable performance in natural language processing tasks. However, the growing number of parameters has resulted in significant increases in memory usage and computational costs [1], [2]. To mitigate these issues, weight quantization techniques have been developed to improve memory efficiency [3], [4]. Despite these advances, there remains a critical need for processors that can efficiently manage floating point (FP) activations and integer (INT) weights [5].

Computing-in-Memory (CIM) is known for its ability to perform energy-efficient INT-INT multiply-accumulate (MAC) operations [6], [7]. Recently, CIM supporting FP-FP MAC operations has also been developed [8], [9], [10]. Although practical AI workloads require FP-INT MAC operations, there are limitations in efficiently implementing these on existing CIM processors. FP-INT CIM processors have been under-explored, leading to efforts to implement FP-INT MAC operations using existing FP-CIM architectures. However, current CIM designs face major challenges in efficiently supporting FP-INT MAC operations. As shown in Fig. 1 (a), traditional FP-CIM performs INT MAC operations by concatenating the sign and mantissa with the hidden '1' and aligning them based on the input group's maximum exponent [9], [10]. This increases input feeding cycles, further exacerbated by the addition of reserved bits (δ) to prevent LSB truncation.

To prevent the increase in input feeding cycles, the CIM for FP-INT MAC can apply post-alignment after mantissa multiplication, similar to conventional digital processing elements (PEs). This reduces feeding cycles and lowers macro activity. However, the adder tree bit-width increases due to the δ-bits, resulting in negligible gains in overall energy consumption. Furthermore, as illustrated in Fig. 1 (b), the adder tree consumes a significant portion of power and area—up to 50.4% and 41.8% in FP8/INT4 MAC operations—which reduces computational efficiency [11], [12].

Additionally, in FP DCIM processors, the accumulator constitutes a major bottleneck, consuming up to 25.9% of the total power. The accumulator comprises an INT accumulator, an FXP-to-FP converter, and an FP accumulator. Both of the FXP2FP converter and the FP accumulator are power-intensive logics and lead to low energy-efficiency [13].

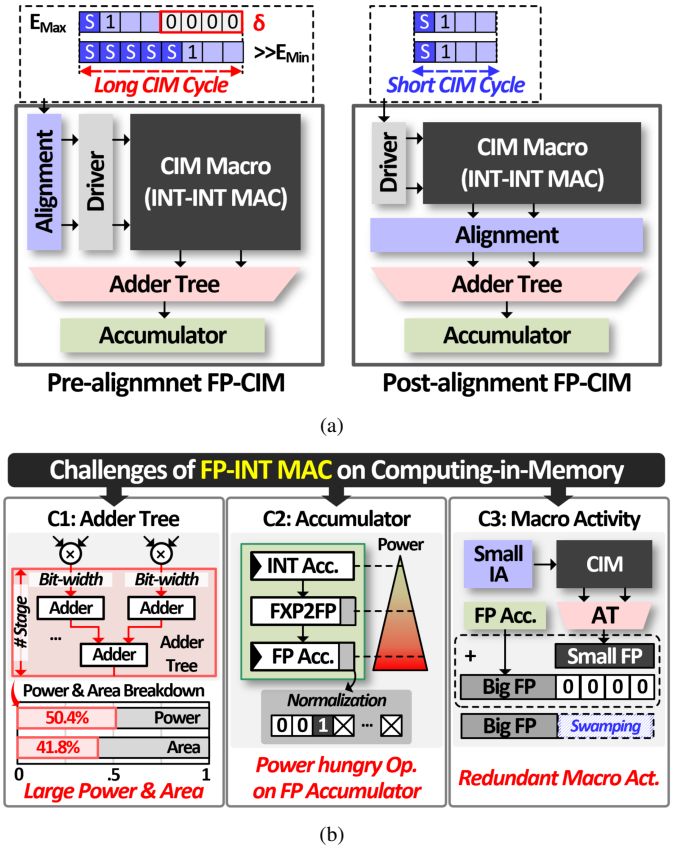


Fig. 1. (a) Pre-alignment and post-alignment FP-CIM. (b) Challenges of previous FP-CIM for FP-INT MAC.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 29,2026 at 12:17:13 UTC from IEEE Xplore. Restrictions apply.

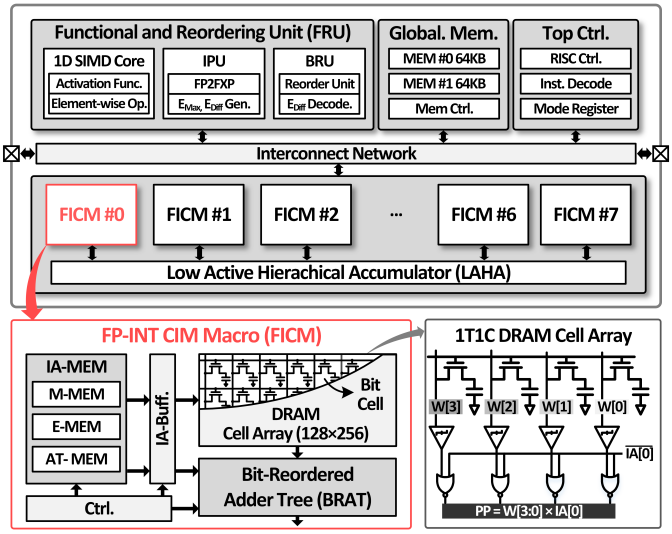


Fig. 2. Overall architecture of FP-INT CIM.

Although post-alignment reduces macro activity, some operations during FP accumulation remain redundant. If the difference between exponents of the stored and newly added FP values exceeds the mantissa bit-width, normalization discards the smaller mantissa (swamping problem), leading to unnecessary macro activity.

This paper proposes a energy-efficient DRAM-based digital computing-in-memory processor optimized for FP-INT operations in data-intensive AI workloads, featuring three key innovations: 1) Bit-reordered adder tree (BRAT) with exponent-aware reordering that reduces the adder tree bit-width to enhance energy-efficiency and area-efficiency by rearranging the serial input sequence; 2) Low active hierarchical accumulator (LAHA) with conditional update, which minimizes power consumption by transferring data to the FP accumulator only when the INT accumulator is full; and 3) Speculative input block skipping (SIBS) that achieves additional power reduction by skipping redundant FP accumulation, leading to an overall power saving of 31%.

II. Overall Architecture

Fig. 2 shows the overall architecture. The chip contains eight FP-INT CIM macros (FICM), a low active hierarchical accumulator (LAHA), a functional and reordering Unit (FRU), global memory, and a top controller.

The FICM consists of an input activation memory (IA-MEM), a 128×256 1T1C DRAM cell array, and BRAT. The IA-MEM is composed of a mantissa memory (M-MEM), an exponent memory (E-MEM), and a BRAT configuration memory (AT-MEM). The M-MEM stores the sign and mantissa of FP input activations in 2's complement format. The E-MEM holds one maximum exponent for each group of 64 bundled inputs. The AT-MEM contains configuration information utilized by the BRAT. The 1T1C DRAM cell array reads 64 INT4 weights from its bit cells and performs bit-wise multiplication with the 64 input mantissas fetched from the M-MEM, then sends the results to the BRAT. The BRAT aligns the 64 4-bit partial sums according to the exponents retrieved from the E-MEM, passes them through an adder tree, and forwards them to LAHA. The LAHA accumulates the INT values received from the BRAT during the input feeding cycles, converts them to FP8, and then performs FP accumulation.

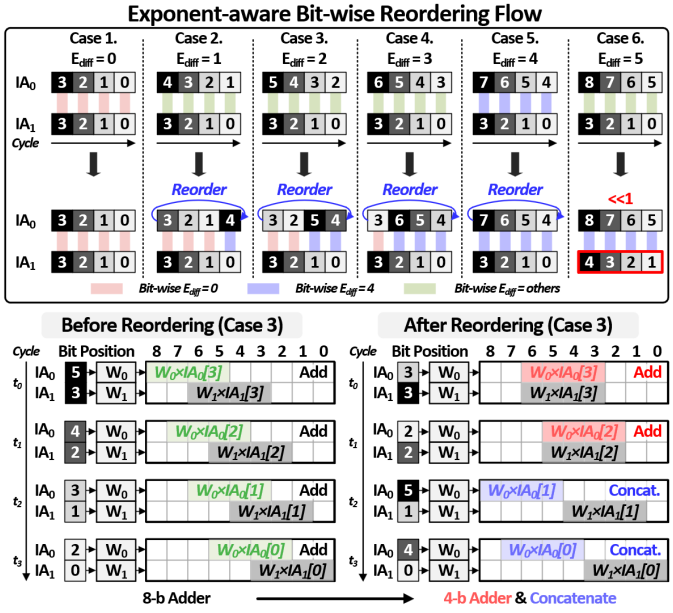


Fig. 3. Partial product exponent-aware BRU operation.

The FRU comprises a 1D SIMD core, an input preprocessing unit (IPU), and a bit-wise reordering unit (BRU). The IPU preprocesses the mantissas and exponents of the output FP8 values after the activation function. The FP2FXP module converts FP8 values into 2's complement format and stores them in global memory (GMEM). For the exponents, it calculates the maximum exponent (E_{Group}) for the 64 inputs, groups the inputs in pairs, computes the larger exponent ($E_{local,max}$) between two FP8 values, and determines the exponent difference (E_{diff}) between the two exponents to send to the BRU. The BRU performs bit reordering based on the E_{diff} calculated by the IPU to facilitate add-and-concatenate operations in the BRAT. It also generates selection signals utilized by the BRAT multiplexer.

III. Bit-Reordered Adder Tree (BRAT) with Exponent-Aware Reordering

The conventional adder tree structure is inefficient in terms of energy and area efficiency because it uses high bit-width adders starting from the first-stage adder, which occupies the largest portion. To improve this, we propose a BRAT that reduces the power and area of the adder tree based on the bit-wise exponent of partial products, simultaneously considering the exponents and bit positions of the weights. In this process, the bit-reordered input mantissas, which are provided to the BRAT in a bit-serial manner, are preprocessed by the BRU and then fetched into IA-MEM during BRAT operations.

A. Exponent-aware Reordering on BRU

Fig. 3 shows how the BRU reorders bits based on the exponent difference E_{diff} between two inputs, IA_0 and IA1.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 29,2026 at 12:17:13 UTC from IEEE Xplore. Restrictions apply.

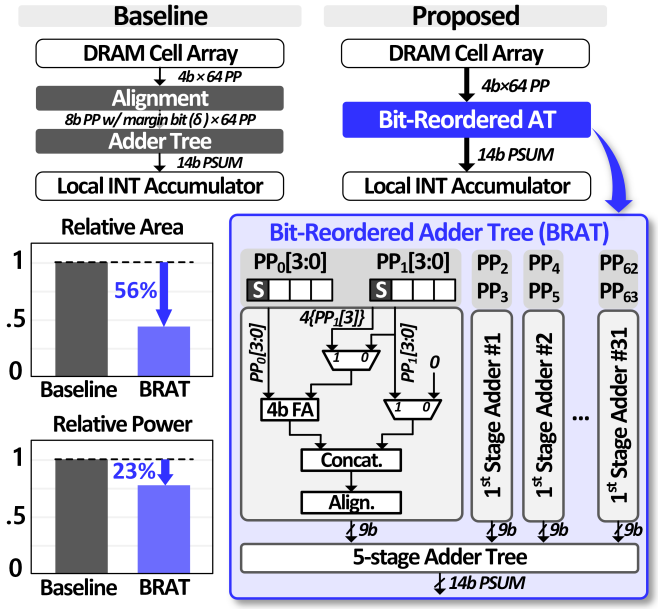


Fig. 4. BRAT architecture.

When the input precision is FP8 and $E_{diff} = 2$ (Case 3), the mantissas of $IA_0[i]$ and $IA_1[i]$ are processed bit-serially through the CIM macro. These mantissas, when multiplied by weights, produce partial products ($PP_n = W_n[3:0] \times IA_n[i]$), typically requiring at least a 6-bit adder. By applying barrel-shift reordering, the bit-serial input is adjusted so that $IA_0[1]$ and $IA_1[3]$ are processed in the same cycle, allowing a 4-bit adder. Before reordering, the bit-wise exponent difference, BE_{diff}, between $IA_0[3]$ and $IA_1[3]$ matches E_{diff}. After reordering, however, the difference between $IA_0[1]$ and $IA_1[3]$ becomes zero. This enables operations to proceed on the same scale. This reordering reduces the bit-width of the adder, allowing efficient computation with a low-bit adder. When $BE_{diff} = 4$, partial products can be concatenated instead of a full addition, further simplifying the operation.

If $E_{diff} \geq 5$, we left-shift the input with the smaller exponent to adjust E_{diff} to 4. Due to the locality characteristics of activations, cases where $E_{diff} \geq 5$ rarely occur, so the impact of reordering method on accuracy is minimal. Therefore, for all E_{diff} cases, we can adjust BE_{diff} to 0 or 4 through reordering, which allows us to avoid using an 8-bit adder.

B. Bit-Reordered Adder Tree (BRAT)

Fig. 4 illustrates the operational flow of the BRAT. Since the BRU adjusts all BE_{diff} to 0 or 4, the BRAT replaces an 8-bit adder with a 4-bit adder that adds effective bits and a concatenate unit. Based on $E_{local,max}$ obtained from the IPU, the input with the larger exponent is fed as PP_{2n}, and the smaller value is fed as PP_{2n+1}.

If $BE_{diff} = 0$, mux selection '0' is activated, and the 4-bit adder adds PP_{2n} and PP_{2n+1}. Four zeros are then padded after the output of the full adder (FA). If $BE_{diff} = 4$, mux selection '1' is activated, and the 4-bit adder adds PP_{2n} and the sign-extended four bits of $PP_{2n+1}[3]$, followed by concatenation of PP_{2n+1} after the output of the FA.

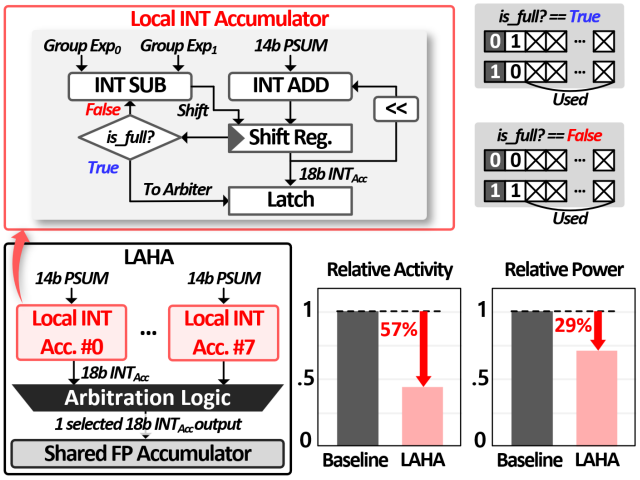


Fig. 5. LAHA architecture and conditional update mechanism.

After concatenation, the result is right-shifted by $E_{Group} - E_{local,max}$ by the alignment unit and then sent to the adder tree. The add-and-concatenate operation first aligns the exponents of the two input values, and then alignment unit aligns these with the group's maximum exponent, allowing the entire group to be processed for INT accumulation in the adder tree. Comparing the proposed BRAT with the baseline adder tree shows a 23% reduction in power and a 56% reduction in area. Additionally, by sharing the shifter across inputs, the area is further reduced, as a separate shifter is no longer needed for each input.

IV. Low Active Hierarchical Accumulator (LAHA) with Conditional Update

If FP accumulation is performed every time after INT accumulation is completed, the system becomes highly power-hungry. Examining the INT accumulator registers shows that the stored values are often much smaller than the register's capacity even after accumulation is complete. Therefore, instead of sending the result to the FP accumulator after every INT accumulation, the values can be accumulated in the INT accumulators until they are full (is_full). The result is sent to the FP accumulator only when the INT accumulator is full, which reduces the activity of the power-hungry FP accumulator.

Fig. 5 shows the proposed LAHA and the conditional update mechanism. LAHA consists of eight local INT accumulators, arbitration logic, and a shared FP accumulator. By monitoring the state of each INT accumulator's shift register, we can determine whether it is full based on the most significant 2 bits (MSB 2b). If the MSB 2b is 00 or 11, the value is smaller than half of the register's representable range. Conversely, if the MSB 2b is 01 or 10, the accumulator is considered full. The shared FP accumulator processes outputs only from local INT accumulators that have reached full capacity ($is_full ==$ True). Since multiple local INT accumulators can become full at the same time, arbitration logic is employed to handle these cases sequentially. Additionally, an overflow counter is used to track whether the accumulation register width can handle the ongoing calculations, detecting overflow before errors occur.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 29,2026 at 12:17:13 UTC from IEEE Xplore. Restrictions apply.

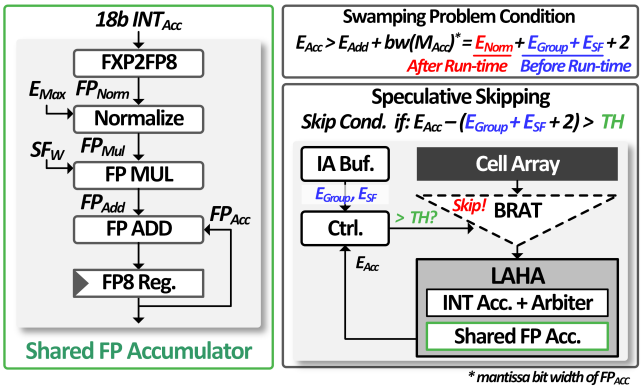
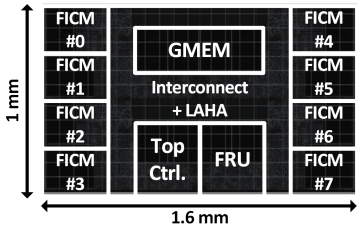


Fig. 6. Speculative input block skipping architecture.



	Specifications
Technology	28nm Logic CMOS
Chip Area	1.6 mm²
Macro Area[1]	0.0825 mm²
Supply Voltage	1.0 V
Clock Frequency	200 MHz
eDRAM CIM Cells	256 Kb
Memory Density[2]	388 Kb/mm²
MAC Density[2] [3]	1.16 TOPS/mm²
Peak Efficiency[3]	64.8 TOPS/W
Retention Time	220 µs

1) FICM 2) Calculated within a macro 3) INT4/FP16 MAC @ 200MHz, 1V

Fig. 7. Layout photograph and performance summary.

With the proposed LAHA and conditional update scheme, the FP accumulator's activity is reduced by 57%, and the power consumption of FP accumulator is decreased by 29%.

V. Speculative Input Block Skipping (SIBS)

Fig. 6 illustrates the shared FP accumulator for SIBS. The 18-bit output from the local accumulator (INT_{Acc}) is converted to FP8 format, with its exponent added with the group's maximum exponent (E_{Group}) and then multiplied by the weight scale factor (SF_W). During FP addition, if the difference between the exponents of FP_{Acc} and FP_{Add} exceeds the mantissa bit width, the smaller mantissa is discarded (swamping problem), leading to redundant computations. By comparing the exponents, speculative skipping avoids such unnecessary operations when the difference exceeds a threshold.

The proposed SIBS reduces unnecessary activity by skipping computations for inputs where the difference between E_{Add} and $(E_{Group} + E_{SF} + bw(M_{Acc}))$ exceeds a predefined threshold. This speculative skipping reduces the macro's activity and improves energy-efficiency by avoiding computations that would not meaningfully impact the final result. The FICM controller receives E_{Acc} from the shared FP accumulator, along with E_{Group} and E_{SF} from the input buffer, and compares these values according to the SIBS. If the difference exceeds the threshold, the BRAT operation is skipped. Through proposed SIBS, during GPT2-Medium inference, the activity within the CIM macro is reduced by 17%, and energy consumption is lowered by 15%, with only negligible increases in perplexity.

VI. Implementation Result

Fig. 7 shows the chip photograph and performance summary. The proposed CIM processor is fabricated using 28nm CMOS technology and occupies a die area of 1.6 mm^2, including 256 Kb DRAM array. The processor achieves a macro-level efficiency of 62.8 TOPS/W at 200MHz and 1.0V for FP8/INT4 precision during GPT2-Small inference. Fig. 8 shows the benchmark results for GPT2-Small and GPT2-Medium on the Wikitext-2 dataset. The proposed processor achieves an perplexity of 35.28pp for GPT2-Small, demonstrating significant power and efficiency improvements. Table 1 shows the performance comparison. Compared to previous CIMs, our processor achieves superior energy-efficiency and FoM, leveraging the BRAT, LAHA and SIBS designs.

	GPT-2-Small	GPT-2-Medium
Task	Text Generation	
Dataset	Wikitext-2	
Precision	FP8 Act. / INT4 Weight	
Ground Truth	29.6pp	21.0pp
Accuracy w/o SIBS	33.4pp	24.1pp
Accuracy w/ SIBS	35.28pp	24.8pp
Energy Efficiency	62.8 TOPS/W	45.0 TOPS/W

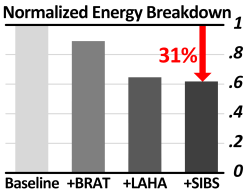


Fig. 8. Benchmark performance of FP-INT CIM.

TABLE I
Comparison Table

	ISSCC'22 [10]	ESSCIRC'23 [12]	ISCAS`23 [14]	TCAS-I`24 [11]	This Work
Technology	28nm	28nm	28nm	55nm	28nm
Area [mm²]	6.69	1.56	2.7	9.8	1.6
Voltage (V)	0.6-1.0	0.55-1.2	0.7-1	1.2	0.8-1
Max Frequency [MHz]	220	650	250	200	200
Memory Type	6T SRAM	8T SRAM	6T SRAM	8T SRAM	1T1C eDRAM
Cell Density[1] [KB/mm²]	1.79	0.7	1.3	6.53	48.5
Peak Energy Efficiency[1] [TOPS/W] or [TFLOPS/W]	46.2 (BF16)	12.1 (FP8)	5.99 (BFP16)	16.6 (INT8)	62.8 (INT4/FP8)
Area Efficiency[1] [TOPS/mm²]	1.15	0.418	0.052	0.072	1.16
FoM[2]	53.13	5.06	0.63	1.2	72.85

1) Calculated within a macro 2) FoM = Peak Energy Efficiency × Area Efficiency

VII. Conclusion

In this paper, we proposed the first energy-efficient DRAM-based digital CIM processor to support FP-INT operations, specifically designed for data-intensive AI workloads. The processor features three key innovations to improve energy-efficiency and reduce area overhead. First, by applying BRAT, the adder bit-width reduced, improving energy-efficiency by 16%. Second, LAHA with conditional update only transfers data to the FP accumulator when the INT accumulator is full, reducing the activity of FP operations by 57%. Third, the SIBS technique skips inputs with significant exponent differences, achieving an additional 17% reduction in CIM macro activity. As a result, the proposed processor achieved a perplexity of 35.28pp in FP8/INT4 precision on the GPT2-Small and delivered a peak performance of 62.8 TOPS/W.

Acknowledgment

This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) under the Graduate School of Artificial Intelligence Semiconductor(IITP-2025-RS-2023-00256472) grant funded by the Korea government(MSIT). The EDA tool was supported by the IC Design Education Center(IDEC), Korea.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 29,2026 at 12:17:13 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] J. Achiam *et al.*, "Gpt-4 technical report," arXiv preprint arXiv:2303.08774, 2023.

[2] A. Dubey *et al.*, "The llama 3 herd of models," arXiv preprint arXiv:2407.21783, 2024.

[3] J. Lin, J. Tang, H. Tang, S. Yang, W. Chen, W. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "AWQ: activation-aware weight quantization for on-device LLM compression and acceleration," in Proceedings of the Seventh Annual Conference on Machine Learning and Systems, MLSys 2024, Santa Clara, CA, USA, May 13-16, 2024 (P. B. Gibbons, G. Pekhimenko, and C. D. Sa, eds.), mlsys.org, 2024. https://doi.org/10.48550/arXiv.2306.00978

[4] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "Gptq: Accurate post-training quantization for generative pre-trained transformers," arXiv preprint arXiv:2210.17323, 2022.

[5] J. Jang, Y. Kim, J. Lee and J. -J. Kim, "FIGNA: Integer Unit-Based Accelerator Design for FP-INT GEMM Preserving Numerical Accuracy," 2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA), Edinburgh, United Kingdom, 2024, pp. 760-773

[6] S. Um, S. Kim, S. Hong, S. Kim and H. -J. Yoo, "LOG-CIM: An Energy-Efficient Logarithmic Quantization Computing-In-Memory Processor With Exponential Parallel Data Mapping and Zero-Aware 6T Dual-WL Cell," in IEEE Journal of Solid-State Circuits, vol. 59, no. 10, pp. 3330-3341, Oct. 2024, doi: 10.1109/JSSC.2024.3399706.

[7] S. Cheon, K. Lee and J. Park, "A 2941-TOPS/W Charge-Domain 10T SRAM Compute-in-Memory for Ternary Neural Network," in IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 70, no. 5, pp. 2085-2097, May 2023, doi: 10.1109/TCSI.2023.3241385.

[8] R. Guo *et al.*, "20.2 A 28nm 74.34TFLOPS/W BF16 Heterogenous CIM-Based Accelerator Exploiting Denoising-Similarity for Diffusion Models," 2024 IEEE International Solid-State Circuits Conference (ISSCC), San Francisco, CA, USA, 2024, pp. 362-364, doi: 10.1109/ISSCC49657.2024.10454308.

[9] Yue *et al.*, "A 28nm 16.9-300TOPS/W Computing-in-Memory Processor Supporting Floating-Point NN Inference/Training with Intensive-CIM Sparse-Digital Architecture," 2023 IEEE International Solid-State Circuits Conference (ISSCC), San Francisco, CA, USA, 2023, pp. 1-3, doi: 10.1109/ISSCC42615.2023.10067779.

[10] F. Tu *et al.*, "A 28nm 29.2TFLOPS/W BF16 and 36.5TOPS/W INT8 Reconfigurable Digital CIM Processor with Unified FP/INT Pipeline and Bitwise In-Memory Booth Multiplication for Cloud Deep Learning Acceleration," 2022 IEEE International Solid-State Circuits Conference (ISSCC), San Francisco, CA, USA, 2022, pp. 1-3, doi: 10.1109/ISSCC42614.2022.9731762.

[11] W. Yi *et al.*, "RDCIM: RISC-V Supported Full-Digital Computing-in-Memory Processor With High Energy Efficiency and Low Area Overhead," in IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 71, no. 4, pp. 1719-1732, April 2024, doi: 10.1109/TCSI.2024.3350664.

[12] J. Saikia, A. Sridharan, I. Yeo, S. Venkataramanaiah, D. Fan and J. -S. Seo, "FP-IMC: A 28nm All-Digital Configurable Floating-Point In-Memory Computing Macro," ESSCIRC 2023- IEEE 49th European Solid State Circuits Conference (ESSCIRC), Lisbon, Portugal, 2023, pp. 405-408, doi: 10.1109/ESSCIRC59616.2023.10268770.

[13] S. R. Vangal, Y. V. Hoskote, N. Y. Borkar and A. Alvandpour, "A 6.2-GFlops Floating-Point Multiply-Accumulator With Conditional Normalization," in IEEE Journal of Solid-State Circuits, vol. 41, no. 10, pp. 2314-2323, Oct. 2006, doi: 10.1109/JSSC.2006.881557.

[14] W. Park *et al.*, "A 5.99 TFLOPS/W Heterogeneous CIM-NPU Architecture for an Energy Efficient Floating-Point DNN Acceleration," 2023 IEEE International Symposium on Circuits and Systems (ISCAS), Monterey, CA, USA, 2023, pp. 1-4, doi: 10.1109/ISCAS46773.2023.10181869.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 29,2026 at 12:17:13 UTC from IEEE Xplore. Restrictions apply.